ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas · Carrera de Ciencia de Datos e Inteligencia Artificial

INFORME DE PRÁCTICA N.º2

Clasificación de variables y población vs. muestra

Tópico 1 del PEA · Asignatura IDSD422 — Estadística y Programación para Ciencia de Datos I

Estudiante(s)	Villacís Vega, Alejandro Xavier — usuario de GitHub: altas123linux
Docente	Ing. Christian Suárez, M.Sc.
Período académico	2026-B
Fecha de entrega	23/10/2026
Repositorio (fork)	https://github.com/altas123linux/IDSD422-lab-muestreo-poblacion.git

Resumen ejecutivo

Clasifiqué las ocho columnas de la base de fidelización siete variables más el identificador donde tres son numéricas edad e ingresos_mensuales continuas, numero_hijos discreta y cinco categóricas cliente_id, género y ciudad nominales; nivel_educacion y satisfaccion ordinales. Luego estimé el ingreso mensual promedio de los 400 clientes con muestras de 20. El parámetro real es 1330.72 USD; la conveniencia estimó 1348.50, el aleatorio simple 1336.00 y el estratificado 1287.50 USD, pero la conveniencia dejó fuera a Guayaquil y Cuenca y cinco sorteos aleatorios variaron entre 1235.00 y 1491.50 USD. Recomiendo estimar con un diseño aleatorio, de preferencia estratificado por ciudad, y reportar la cifra con su incertidumbre.

1. Introducción

RetailAndina S.A., cadena ecuatoriana de consumo masivo, pidió al equipo de BI dos cosas: conocer por dentro su base de fidelización (qué variables tiene y cuáles se pueden promediar, ordenar o solo contar) y estimar el ingreso mensual promedio de sus clientes con muestras de 20, demostrando que esas muestras representan a los 400 registros de la base. Como analista de BI clasifiqué las variables verificando cada decisión en código y comparé tres diseños de muestreo contra el parámetro poblacional. Esto importa porque una variable mal clasificada lleva a cálculos sin sentido, como promediar una escala de satisfacción, y porque una muestra mal diseñada da una respuesta equivocada que no se arregla con más filas.

2. Objetivos

Objetivo general. Clasificar las variables de la base de fidelización y comparar tres diseños de muestreo contra el parámetro poblacional para explicar de qué depende la representatividad de una muestra.

1. Clasificar las siete variables y el identificador de clientes.csv con el criterio de continuidad y orden, y verificar el tipo con dtypes/str() y los valores únicos.
2. Declarar el orden de las ordinales con CategoricalDtype y con factor ordenado en R, y comprobarlo con min().
3. Implementar los diseños por conveniencia, aleatorio simple (cinco semillas) y estratificado por ciudad, y comparar sus medias con el parámetro.

3. Desarrollo de la práctica

Trabajé sobre mi fork del repositorio y completé los TODO en orden: primero clasificacion_variables y después muestreo. Corrí la versión en Python en Google Colab y la versión en R en Posit Cloud, con los archivos de data/ cargados en cada entorno.

3.1 Parte A · Clasificación de las variables

Paso 1. Estructura, tipos inferidos y valores únicos

Cargué data/clientes.csv (**12 filas × 8 columnas**) e imprimí clientes.dtypes. Llegaron como int64 edad, numero_hijos, ingresos_mensuales y satisfaccion, y como object (texto) cliente_id, genero, nivel_educacion y ciudad. Para el TODO 2 escribí un bucle que muestra los valores únicos de cada columna:

```
clientes = pd.read_csv("data/clientes.csv")
print(clientes.dtypes)              # tipo inferido: solo una pista
for columna in clientes.columns:    # valores únicos por columna
    u = clientes[columna].unique()
    print(f"{columna}: {len(u)} únicos -> {list(u)}")
```

El bucle mostró que cliente_id tiene 12 únicos en 12 filas un identificador, numero_hijos 5 0 a 4, genero 3 Femenino, Masculino, Otro: nominal, no binaria, nivel_educacion 4, ciudad 3 y satisfaccion 4 2 a 5: nadie respondió 1 En Colab los números salieron como np.int64(34); es solo cómo NumPy 2 imprime los enteros, y en el script del repositorio lo limpié con .item().

Paso 2. Declaración del orden de las ordinales

Una ordinal sin orden declarado se ordena alfabéticamente Posgrado antes que Primaria. Para los TODO 3 y 4 usé CategoricalDtype:

```
orden_edu = pd.CategoricalDtype(categories=["Primaria", "Secundaria",
                                "Universidad", "Posgrado"], ordered=True)
clientes["nivel_educacion"] = clientes["nivel_educacion"].astype(orden_edu)
orden_sat = pd.CategoricalDtype(categories=[1, 2, 3, 4, 5], ordered=True)
clientes["satisfaccion"] = clientes["satisfaccion"].astype(orden_sat)
print(clientes["nivel_educacion"].min(), clientes["satisfaccion"].min())
```



Figura 1. Colab: carga de clientes.csv, dtypes y bucle de valores únicos en python

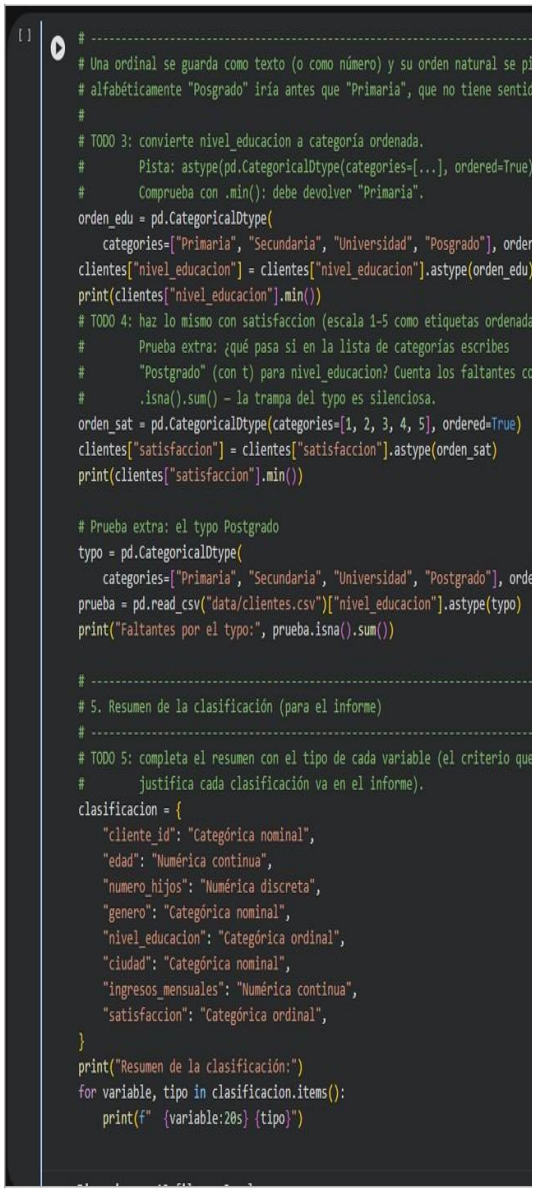
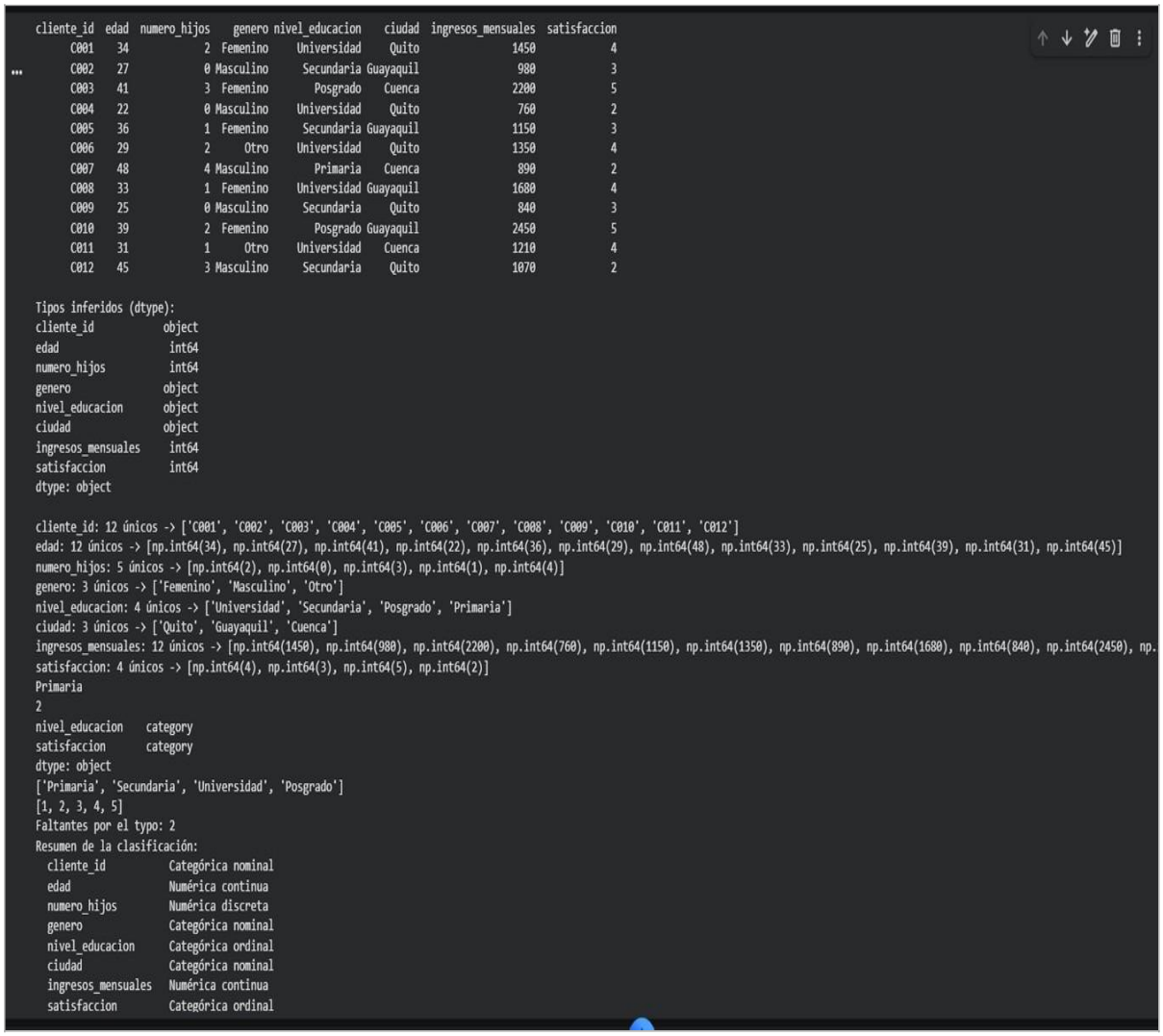


Figura 2. Colab: orden declarado con CategoricalDtype, prueba del typo Postgrado y resumen de la clasificación.

La salida (Figura 3) confirmó que min() devolvió **Primaria** y **2**, y que ambas columnas pasaron a dtype category. En la prueba extra escribí «Postgrado» (con t) y .isna().sum() dio **2 faltantes**: los dos clientes con Posgrado (C003 y C010) se volvieron NaN sin ningún error ni aviso. La trampa del typo es silenciosa y solo se detecta contando faltantes después de convertir.



```
  cliente_id  edad  numero_hijos    genero nivel_educacion    ciudad  ingresos_mensuales  satisfaccion
       C001    34             2  Femenino     Universidad     Quito                1450             4
       C002    27             0 Masculino      Secundaria Guayaquil                 980             3
       C003    41             3  Femenino       Posgrado    Cuenca                2200             5
       C004    22             0 Masculino     Universidad     Quito                 760             2
       C005    36             1  Femenino      Secundaria Guayaquil                1150             3
       C006    29             2      Otro     Universidad     Quito                1350             4
       C007    48             4 Masculino       Primaria    Cuenca                 890             2
       C008    33             1  Femenino     Universidad Guayaquil                1680             4
       C009    25             0 Masculino      Secundaria     Quito                 840             3
       C010    39             2  Femenino       Posgrado Guayaquil                2450             5
       C011    31             1      Otro     Universidad    Cuenca                1210             4
       C012    45             3 Masculino      Secundaria     Quito                1070             2

Tipos inferidos (dtype):
cliente_id           object
edad                  int64
numero_hijos          int64
genero               object
nivel_educacion      object
ciudad               object
ingresos_mensuales    int64
satisfaccion          int64
dtype: object

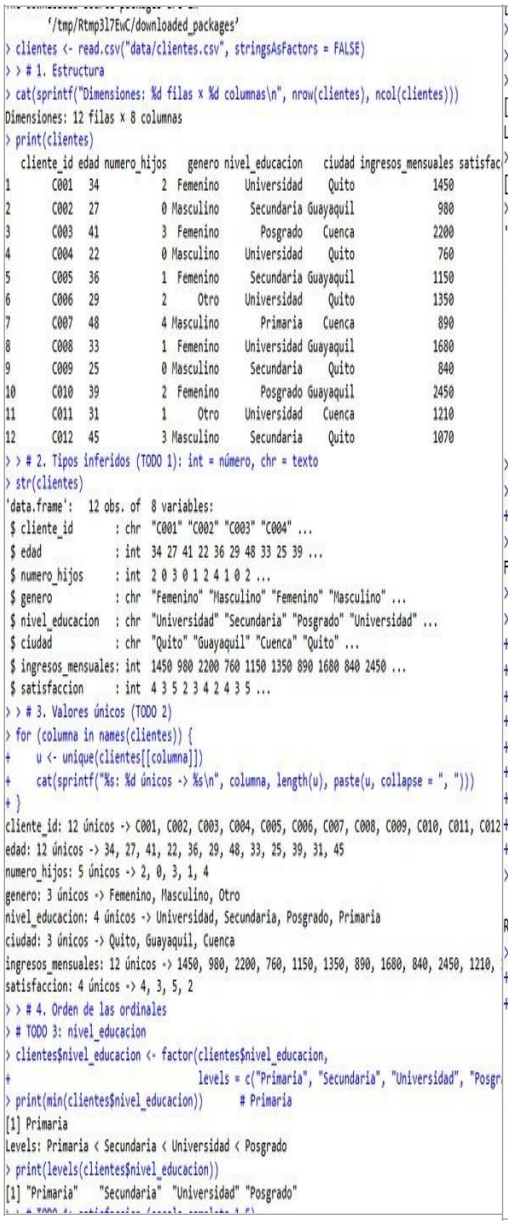
cliente_id: 12 únicos -> ['C001', 'C002', 'C003', 'C004', 'C005', 'C006', 'C007', 'C008', 'C009', 'C010', 'C011', 'C012']
edad: 12 únicos -> [np.int64(34), np.int64(27), np.int64(41), np.int64(22), np.int64(36), np.int64(29), np.int64(48), np.int64(33), np.int64(25), np.int64(39), np.int64(31), np.int64(45)]
numero_hijos: 5 únicos -> [np.int64(2), np.int64(0), np.int64(3), np.int64(1), np.int64(4)]
genero: 3 únicos -> ['Femenino', 'Masculino', 'Otro']
nivel_educacion: 4 únicos -> ['Universidad', 'Secundaria', 'Posgrado', 'Primaria']
ciudad: 3 únicos -> ['Quito', 'Guayaquil', 'Cuenca']
ingresos_mensuales: 12 únicos -> [np.int64(1450), np.int64(980), np.int64(2200), np.int64(760), np.int64(1150), np.int64(1350), np.int64(890), np.int64(1680), np.int64(840), np.int64(2450), np.
satisfaccion: 4 únicos -> [np.int64(4), np.int64(3), np.int64(5), np.int64(2)]
Primaria
2
nivel_educacion    category
satisfaccion       category
dtype: object
['Primaria', 'Secundaria', 'Universidad', 'Posgrado']
[1, 2, 3, 4, 5]
Faltantes por el typo: 2
Resumen de la clasificación:
  cliente_id          Categórica nominal
  edad                Numérica continua
  numero_hijos        Numérica discreta
  genero              Categórica nominal
  nivel_educacion     Categórica ordinal
  ciudad              Categórica nominal
  ingresos_mensuales  Numérica continua
  satisfaccion        Categórica ordinal
```

Figura 3. Salida de la Parte A en Colab: datos, dtypes, valores únicos, mínimos de las ordinales, 2 faltantes por el typo y resumen.

Paso 3. Verificación en R (Posit Cloud)

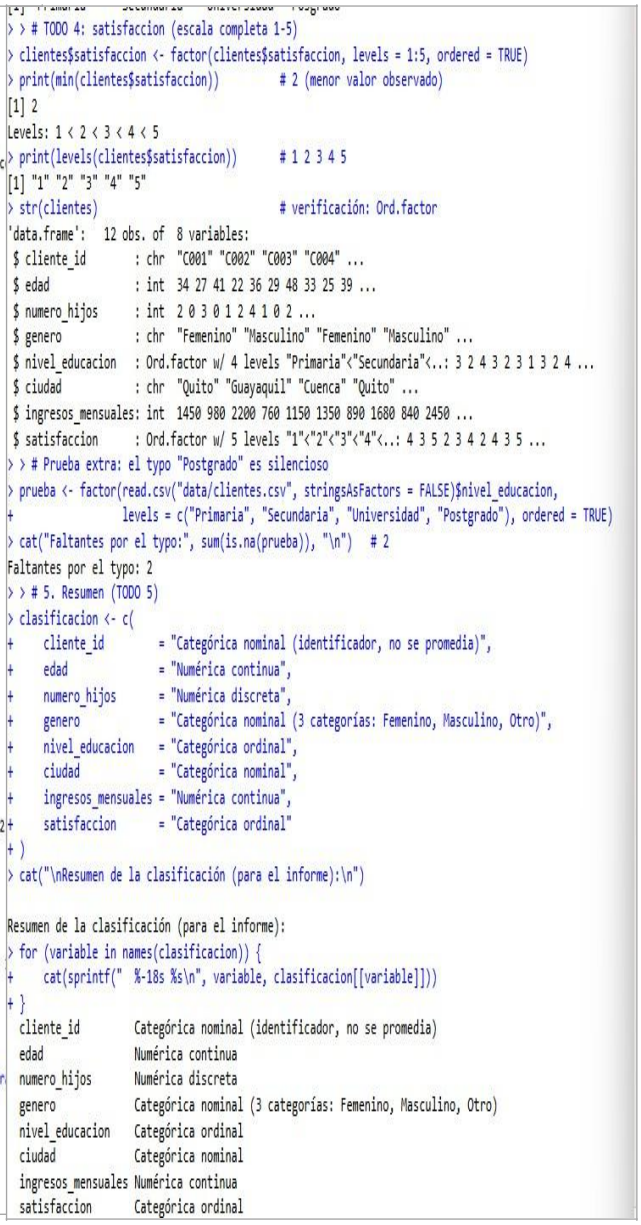
En R, str() mostró la misma división (int y chr). Declaré las ordinales con factor y R imprimió Primaria < Secundaria < Universidad < Posgrado y 1

< 2 < 3 < 4 < 5; el segundo str() mostró Ord.factor 4 levels y Ord.factor 5 levels. El typo también produjo 2 NA.



```
'/tmp/Rtmp3l7EwC/downloaded_packages'
> clientes <- read.csv("data/clientes.csv", stringsAsFactors = FALSE)
> > # 1. Estructura
> cat(sprintf("Dimensiones: %d filas x %d columnas\n", nrow(clientes), ncol(clientes)))
Dimensiones: 12 filas x 8 columnas
> print(clientes)
   cliente_id edad numero_hijos    genero nivel_educacion    ciudad ingresos_mensuales satisfac
1        C001   34            2  Femenino     Universidad     Quito             1450
2        C002   27            0 Masculino      Secundaria Guayaquil              980
3        C003   41            3  Femenino       Posgrado    Cuenca             2200
4        C004   22            0 Masculino     Universidad     Quito              760
5        C005   36            1  Femenino      Secundaria Guayaquil             1150
6        C006   29            2      Otro     Universidad     Quito             1350
7        C007   48            4 Masculino       Primaria    Cuenca              890
8        C008   33            1  Femenino     Universidad Guayaquil             1680
9        C009   25            0 Masculino      Secundaria     Quito              840
10       C010   39            2  Femenino       Posgrado Guayaquil             2450
11       C011   31            1      Otro     Universidad    Cuenca             1210
12       C012   45            3 Masculino      Secundaria     Quito             1070
> > # 2. Tipos inferidos (TODO 1): int = número, chr = texto
> str(clientes)
'data.frame':   12 obs. of  8 variables:
 $ cliente_id       : chr  "C001" "C002" "C003" "C004" ...
 $ edad             : int  34 27 41 22 36 29 48 33 25 39 ...
 $ numero_hijos     : int  2 0 3 0 1 2 4 1 0 2 ...
 $ genero           : chr  "Femenino" "Masculino" "Femenino" "Masculino" ...
 $ nivel_educacion  : chr  "Universidad" "Secundaria" "Posgrado" "Universidad" ...
 $ ciudad           : chr  "Quito" "Guayaquil" "Cuenca" "Quito" ...
 $ ingresos_mensuales: int  1450 980 2200 760 1150 1350 890 1680 840 2450 ...
 $ satisfaccion     : int  4 3 5 2 3 4 2 4 3 5 ...
> > # 3. Valores únicos (TODO 2)
> for (columna in names(clientes)) {
+     u <- unique(clientes[[columna]])
+     cat(sprintf("%s: %d únicos -> %s\n", columna, length(u), paste(u, collapse = ", ")))
+ }
cliente_id: 12 únicos -> C001, C002, C003, C004, C005, C006, C007, C008, C009, C010, C011, C012
edad: 12 únicos -> 34, 27, 41, 22, 36, 29, 48, 33, 25, 39, 31, 45
numero_hijos: 5 únicos -> 2, 0, 3, 1, 4
genero: 3 únicos -> Femenino, Masculino, Otro
nivel_educacion: 4 únicos -> Universidad, Secundaria, Posgrado, Primaria
ciudad: 3 únicos -> Quito, Guayaquil, Cuenca
ingresos_mensuales: 12 únicos -> 1450, 980, 2200, 760, 1150, 1350, 890, 1680, 840, 2450, 1210,
satisfaccion: 4 únicos -> 4, 3, 5, 2
> > # 4. Orden de las ordinales
> # TODO 3: nivel_educacion
> clientes$nivel_educacion <- factor(clientes$nivel_educacion,
+                                    levels = c("Primaria", "Secundaria", "Universidad", "Posgr
> print(min(clientes$nivel_educacion))      # Primaria
[1] Primaria
Levels: Primaria < Secundaria < Universidad < Posgrado
> print(levels(clientes$nivel_educacion))
[1] "Primaria"    "Secundaria"  "Universidad" "Posgrado"
```

Figura 4. Posit Cloud: str(), valores únicos y factor ordenado de nivel_educacion (Parte A, R).



```
> > # TODO 4: satisfaccion (escala completa 1-5)
> clientes$satisfaccion <- factor(clientes$satisfaccion, levels = 1:5, ordered = TRUE)
> print(min(clientes$satisfaccion))         # 2 (menor valor observado)
[1] 2
Levels: 1 < 2 < 3 < 4 < 5
> print(levels(clientes$satisfaccion))      # 1 2 3 4 5
[1] "1" "2" "3" "4" "5"
> str(clientes)                             # verificación: Ord.factor
'data.frame':   12 obs. of  8 variables:
 $ cliente_id       : chr  "C001" "C002" "C003" "C004" ...
 $ edad             : int  34 27 41 22 36 29 48 33 25 39 ...
 $ numero_hijos     : int  2 0 3 0 1 2 4 1 0 2 ...
 $ genero           : chr  "Femenino" "Masculino" "Femenino" "Masculino" ...
 $ nivel_educacion  : Ord.factor w/ 4 levels "Primaria"<"Secundaria"<..: 3 2 4 3 2 3 1 3 2 4 ...
 $ ciudad           : chr  "Quito" "Guayaquil" "Cuenca" "Quito" ...
 $ ingresos_mensuales: int  1450 980 2200 760 1150 1350 890 1680 840 2450 ...
 $ satisfaccion     : Ord.factor w/ 5 levels "1"<"2"<"3"<"4"<..: 4 3 5 2 3 4 2 4 3 5 ...
> > # Prueba extra: el typo "Postgrado" es silencioso
> prueba <- factor(read.csv("data/clientes.csv", stringsAsFactors = FALSE)$nivel_educacion,
+                  levels = c("Primaria", "Secundaria", "Universidad", "Postgrado"), ordered = TRUE)
> cat("Faltantes por el typo:", sum(is.na(prueba)), "\n")   # 2
Faltantes por el typo: 2
> > # 5. Resumen (TODO 5)
> clasificacion <- c(
+     cliente_id        = "Categórica nominal (identificador, no se promedia)",
+     edad              = "Numérica continua",
+     numero_hijos      = "Numérica discreta",
+     genero            = "Categórica nominal (3 categorías: Femenino, Masculino, Otro)",
+     nivel_educacion   = "Categórica ordinal",
+     ciudad            = "Categórica nominal",
+     ingresos_mensuales = "Numérica continua",
+     satisfaccion      = "Categórica ordinal"
+ )
> cat("\nResumen de la clasificación (para el informe):\n")

Resumen de la clasificación (para el informe):
> for (variable in names(clasificacion)) {
+     cat(sprintf("  %-18s %s\n", variable, clasificacion[[variable]]))
+ }
  cliente_id        Categórica nominal (identificador, no se promedia)
  edad              Numérica continua
  numero_hijos      Numérica discreta
  genero            Categórica nominal (3 categorías: Femenino, Masculino, Otro)
  nivel_educacion   Categórica ordinal
  ciudad            Categórica nominal
  ingresos_mensuales Numérica continua
  satisfaccion      Categórica ordinal
```

Figura 5. Posit Cloud: satisfaccion como factor ordenado 1–5, Ord.factor en str(), 2 NA por el typo y resumen.

Paso 4. Tabla de clasificación

Tabla 1. Clasificación de las variables de clientes.csv

Variable	Tipo	Justificación (criterio)	Verificación en código

cliente_id	Categórica nominal (identificador)	Etiqueta que distingue a cada cliente; no se promedia ni hay orden con significado.	object/chr; 12 únicos en 12 filas.
edad	Numérica continua	El tiempo vivido es continuo (34,5 años tiene sentido); llega entera solo porque se registra en años cumplidos.	int64/int; 12 únicos (22 a 48).
numero_hijos	Numérica discreta	Se cuenta; solo toma enteros, no existe medio hijo.	int64/int; 5 únicos (0 a 4).
genero	Categórica nominal	Tres categorías sin orden; al ser tres no es binaria.	object/chr; 3 únicos.
nivel_educacion	Categórica ordinal	Orden natural Primaria < Secundaria < Universidad < Posgrado; la distancia entre niveles no se mide.	object/chr; 4 únicos; orden declarado, min() = Primaria.
ciudad	Categórica nominal	Quito, Guayaquil y Cuenca son grupos sin orden.	object/chr; 3 únicos.

Variable	Tipo	Justificación (criterio)	Verificación en código
ingresos_mensuales	Numérica continua	Monto en USD: la mitad tiene sentido y se puede promediar.	int64/int; 12 únicos (760 a 2450).
satisfaccion	Categórica ordinal	Escala 1–5 con orden, pero un 4 no es el doble de un 2 ni los saltos son iguales.	int64/int (engañoso); 4 únicos; declarada 1–5, min() = 2.

La que más me hizo dudar fue edad: llega en enteros como el número de hijos, pero los hijos se cuentan y la edad mide un tiempo continuo que redondeamos a años cumplidos, por eso la dejé como continua.

P1. ¿Qué te dijo el dtype de satisfaccion y por qué no basta? ¿Qué pasa si una ordinal queda como texto sin declarar el orden?

El dtype fue int64 en Python e int en R: los dos lenguajes la leyeron como número. Pero el dtype solo dice cómo se guardó el dato. Con el criterio, la satisfacción es ordinal: hay orden, pero no tiene sentido decir que un 4 es el doble de un 2, así que su media no tendría un significado real. Si una ordinal queda como texto sin orden declarado, se ordena alfabéticamente y .min() o una comparación dan resultados sin sentido; tras declararla, min() devolvió Primaria y 2.

3.2 Parte B · Población vs. muestra

Paso 1. La población y su parámetro

Cargué data/clientes_400.csv (**400 clientes × 8 columnas**): Quito 200 (50 %), Guayaquil 120 (30 %) y Cuenca 80 (20 %). Con poblacion["ingresos_mensuales"].mean() obtuve **1330.72 USD**, igual que con mean() en R.

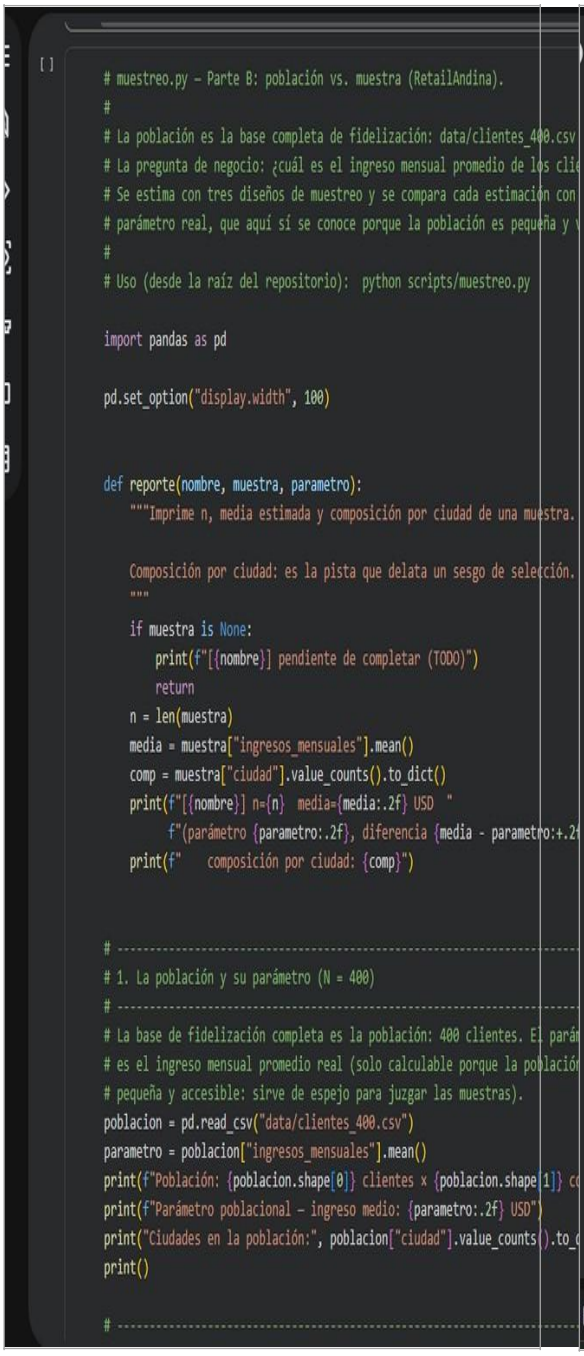
Tabla 2. Población, variable de interés y parámetro

Concepto	Valor
Población (N)	N = 400 clientes (base de fidelización completa)
Variable de interés	ingresos_mensuales (USD)
Parámetro poblacional (media)	1330.72 USD
¿Por qué se muestrea si el parámetro existe?	En la práctica medir a todos los clientes es caro o imposible. Aquí el parámetro se conoce solo porque la base es pequeña y sirve de espejo para juzgar cada diseño; en un estudio real solo tendríamos la muestra.

Paso 2. Los tres diseños en Python

Tomé muestras de 20 clientes (5 % de la base).

```
muestra_conv = poblacion.head(20)                    # conveniencia
muestra_aas = poblacion.sample(20, random_state=1)   # aleatorio simple
for semilla in range(1, 6):                          # 5 sorteos
    m = poblacion.sample(20, random_state=semilla)
    medias[semilla] = round(m["ingresos_mensuales"].mean(), 2)
muestra_est = poblacion.groupby("ciudad").sample(frac=0.05, random_state=1)
```



```
# muestreo.py – Parte B: población vs. muestra (RetailAndina).
#
# La población es la base completa de fidelización: data/clientes_400.csv
# La pregunta de negocio: ¿cuál es el ingreso mensual promedio de los clie
# Se estima con tres diseños de muestreo y se compara cada estimación con
# parámetro real, que aquí sí se conoce porque la población es pequeña y
#
# Uso (desde la raíz del repositorio):  python scripts/muestreo.py

import pandas as pd

pd.set_option("display.width", 100)

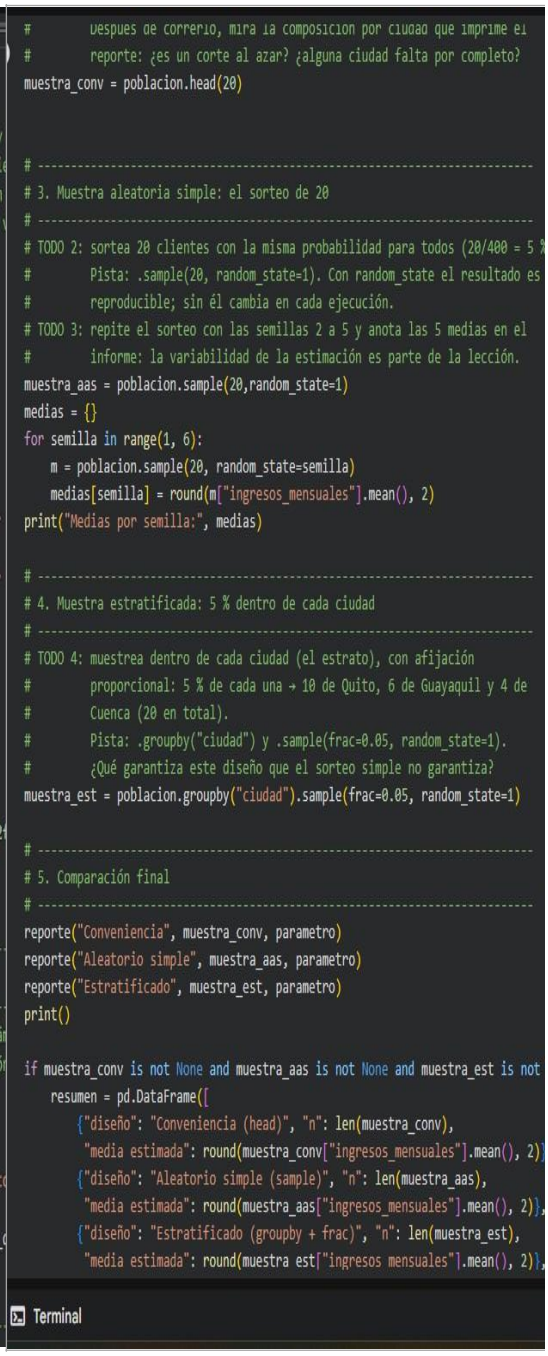

def reporte(nombre, muestra, parametro):
    """Imprime n, media estimada y composición por ciudad de una muestra.

    Composición por ciudad: es la pista que delata un sesgo de selección.
    """
    if muestra is None:
        print(f"[{nombre}] pendiente de completar (TODO)")
        return
    n = len(muestra)
    media = muestra["ingresos_mensuales"].mean()
    comp = muestra["ciudad"].value_counts().to_dict()
    print(f"[{nombre}] n={n}  media={media:.2f} USD  "
          f"(parámetro {parametro:.2f}, diferencia {media - parametro:+.2
    print(f"    composición por ciudad: {comp}")


# ---------------------------------------------------------------------
# 1. La población y su parámetro (N = 400)
# ---------------------------------------------------------------------
# La base de fidelización completa es la población: 400 clientes. El parám
# es el ingreso mensual promedio real (solo calculable porque la població
# pequeña y accesible: sirve de espejo para juzgar las muestras).
poblacion = pd.read_csv("data/clientes_400.csv")
parametro = poblacion["ingresos_mensuales"].mean()
print(f"Población: {poblacion.shape[0]} clientes x {poblacion.shape[1]} c
print(f"Parámetro poblacional – ingreso medio: {parametro:.2f} USD")
print("Ciudades en la población:", poblacion["ciudad"].value_counts().to_
print()

# ---------------------------------------------------------------------
```

Figura 6. Colab: función reporte y cálculo del parámetro poblacional (Parte B, Python).



```
#        Después de correrlo, mira la composición por ciudad que imprime el
#        reporte: ¿es un corte al azar? ¿alguna ciudad falta por completo?
muestra_conv = poblacion.head(20)


# ---------------------------------------------------------------------
# 3. Muestra aleatoria simple: el sorteo de 20
# ---------------------------------------------------------------------
# TODO 2: sortea 20 clientes con la misma probabilidad para todos (20/400 = 5 %
#        Pista: .sample(20, random_state=1). Con random_state el resultado es
#        reproducible; sin él cambia en cada ejecución.
# TODO 3: repite el sorteo con las semillas 2 a 5 y anota las 5 medias en el
#        informe: la variabilidad de la estimación es parte de la lección.
muestra_aas = poblacion.sample(20,random_state=1)
medias = {}
for semilla in range(1, 6):
    m = poblacion.sample(20, random_state=semilla)
    medias[semilla] = round(m["ingresos_mensuales"].mean(), 2)
print("Medias por semilla:", medias)

# ---------------------------------------------------------------------
# 4. Muestra estratificada: 5 % dentro de cada ciudad
# ---------------------------------------------------------------------
# TODO 4: muestrea dentro de cada ciudad (el estrato), con afijación
#        proporcional: 5 % de cada una → 10 de Quito, 6 de Guayaquil y 4 de
#        Cuenca (20 en total).
#        Pista: .groupby("ciudad") y .sample(frac=0.05, random_state=1).
#        ¿Qué garantiza este diseño que el sorteo simple no garantiza?
muestra_est = poblacion.groupby("ciudad").sample(frac=0.05, random_state=1)

# ---------------------------------------------------------------------
# 5. Comparación final
# ---------------------------------------------------------------------
reporte("Conveniencia", muestra_conv, parametro)
reporte("Aleatorio simple", muestra_aas, parametro)
reporte("Estratificado", muestra_est, parametro)
print()

if muestra_conv is not None and muestra_aas is not None and muestra_est is not
    resumen = pd.DataFrame([
        {"diseño": "Conveniencia (head)", "n": len(muestra_conv),
         "media estimada": round(muestra_conv["ingresos_mensuales"].mean(), 2)
        {"diseño": "Aleatorio simple (sample)", "n": len(muestra_aas),
         "media estimada": round(muestra_aas["ingresos_mensuales"].mean(), 2)},
        {"diseño": "Estratificado (groupby + frac)", "n": len(muestra_est),
         "media estimada": round(muestra_est["ingresos_mensuales"].mean(), 2)},
```

Terminal

Figura 7. Colab: los tres diseños (head, sample con semillas 1–5, groupby + sample) y la comparación final.

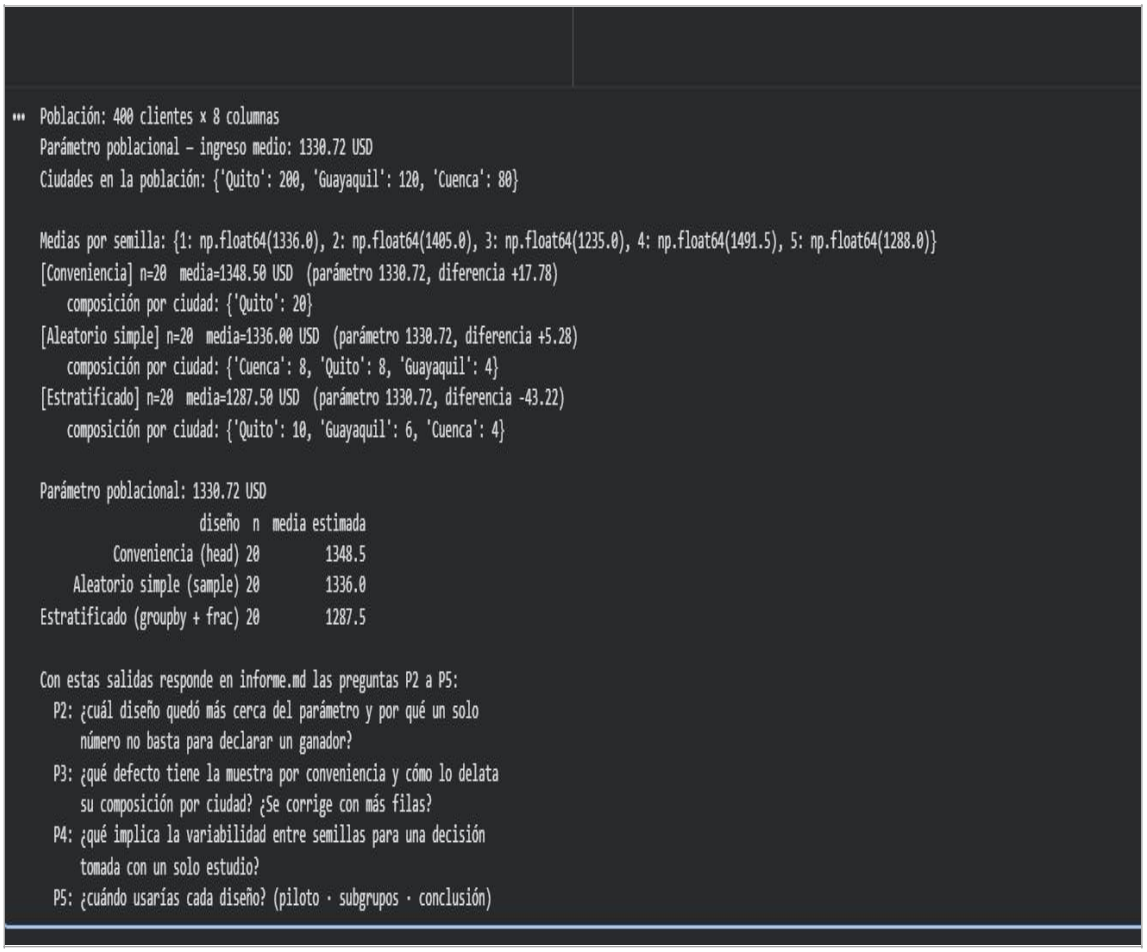


Figura 8. Salida de la Parte B en Colab: parámetro, medias por semilla y n, media, diferencia y composición de cada diseño.

Tabla 3. Resultados de los tres diseños (Python, semilla 1)

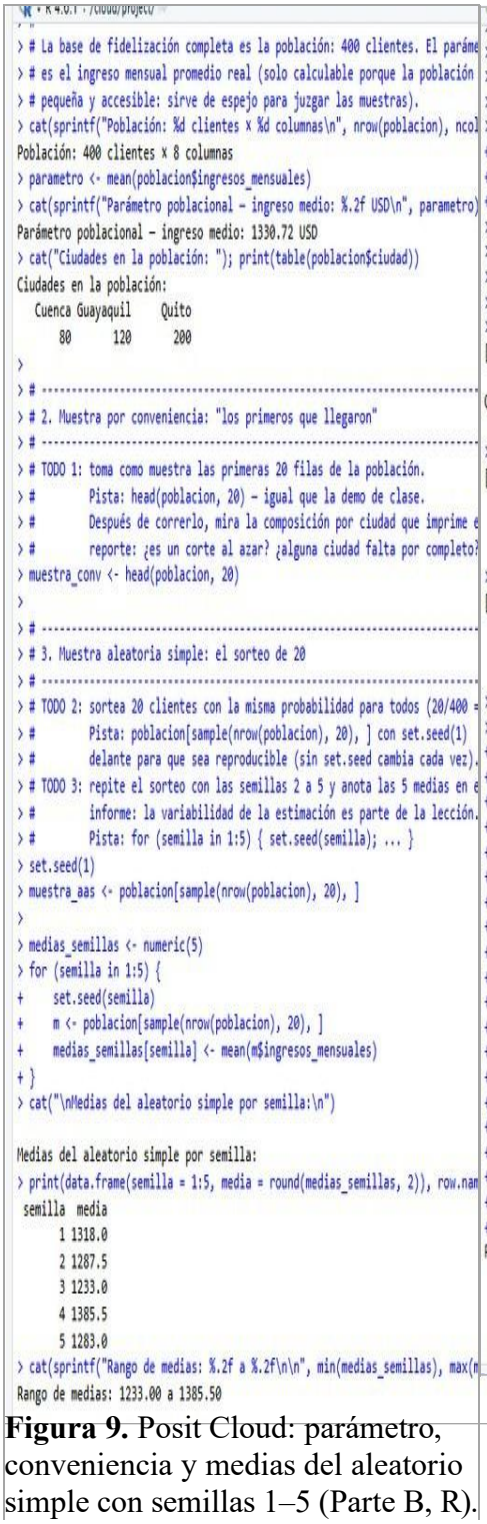
Diseño	n	Media estimada	Composición por ciudad	Diferencia vs. parámetro
Conveniencia (head)	20	1348.50 USD	Quito 20	+17.78
Aleatorio simple (sample)	20	1336.00 USD	Cuenca 8, Quito 8, Guayaquil 4	+5.28
Estratificado (groupby + 5 %)	20	1287.50 USD	Quito 10, Guayaquil 6, Cuenca 4	−43.22

Tabla 4. Medias del aleatorio simple con semillas 1 a 5 (USD), en Python y en R

Semilla	1	2	3	4	5	Rango
Python	1336.00	1405.00	1235.00	1491.50	1288.00	256.50
R	1318.00	1287.50	1233.00	1385.50	1283.00	152.50

Paso 3. Los tres diseños en R (Posit Cloud)

En R usé head(poblacion, 20), poblacion[sample(nrow(poblacion), 20),] con set.seed() y group_by(ciudad) |> slice_sample(prop = 0.05). La conveniencia dio lo mismo que en Python (1348.50 USD, Quito 20) porque head no sortea. El aleatorio simple con semilla 1 dio **1318.00 USD** (−12.72; Cuenca 5, Guayaquil 7, Quito 8) y el estratificado **1226.00 USD** (−104.72; Cuenca 4, Guayaquil 6, Quito 10). Con la misma semilla R y Python no eligen los mismos clientes porque cada uno tiene su propio generador aleatorio; la semilla solo asegura repetir el resultado dentro del mismo lenguaje.



```
> # La base de fidelización completa es la población: 400 clientes. El paráme
> # es el ingreso mensual promedio real (solo calculable porque la población
> # pequeña y accesible: sirve de espejo para juzgar las muestras).
> cat(sprintf("Población: %d clientes x %d columnas\n", nrow(poblacion), ncol
Población: 400 clientes x 8 columnas
> parametro <- mean(poblacion$ingresos_mensuales)
> cat(sprintf("Parámetro poblacional – ingreso medio: %.2f USD\n", parametro)
Parámetro poblacional – ingreso medio: 1330.72 USD
> cat("Ciudades en la población: "); print(table(poblacion$ciudad))
Ciudades en la población:
   Cuenca Guayaquil     Quito
       80       120       200
>
> # ------------------------------------------------------------------
> # 2. Muestra por conveniencia: "los primeros que llegaron"
> # ------------------------------------------------------------------
> # TODO 1: toma como muestra las primeras 20 filas de la población.
> #         Pista: head(poblacion, 20) – igual que la demo de clase.
> #         Después de correrlo, mira la composición por ciudad que imprime e
> #         reporte: ¿es un corte al azar? ¿alguna ciudad falta por completo?
> muestra_conv <- head(poblacion, 20)
>
> # ------------------------------------------------------------------
> # 3. Muestra aleatoria simple: el sorteo de 20
> # ------------------------------------------------------------------
> # TODO 2: sortea 20 clientes con la misma probabilidad para todos (20/400 =
> #         Pista: poblacion[sample(nrow(poblacion), 20), ] con set.seed(1)
> #         delante para que sea reproducible (sin set.seed cambia cada vez).
> # TODO 3: repite el sorteo con las semillas 2 a 5 y anota las 5 medias en e
> #         informe: la variabilidad de la estimación es parte de la lección.
> #         Pista: for (semilla in 1:5) { set.seed(semilla); ... }
> set.seed(1)
> muestra_aas <- poblacion[sample(nrow(poblacion), 20), ]
>
> medias_semillas <- numeric(5)
> for (semilla in 1:5) {
+     set.seed(semilla)
+     m <- poblacion[sample(nrow(poblacion), 20), ]
+     medias_semillas[semilla] <- mean(m$ingresos_mensuales)
+ }
> cat("\nMedias del aleatorio simple por semilla:\n")

Medias del aleatorio simple por semilla:
> print(data.frame(semilla = 1:5, media = round(medias_semillas, 2)), row.nam
 semilla  media
       1 1318.0
       2 1287.5
       3 1233.0
       4 1385.5
       5 1283.0
> cat(sprintf("Rango de medias: %.2f a %.2f\n\n", min(medias_semillas), max(m
Rango de medias: 1233.00 a 1385.50
```

Figura 9. Posit Cloud: parámetro, conveniencia y medias del aleatorio simple con semillas 1–5 (Parte B, R).

```
> #         ¿Qué garantiza este diseño que el sorteo simple no garantiza?
> suppressPackageStartupMessages(library(dplyr))
> set.seed(1)
> muestra_est <- poblacion |>
+     group_by(ciudad) |>
+     slice_sample(prop = 0.05) |>
+     ungroup()
>
> # ------------------------------------------------------------------
> # 5. Comparación final
> # ------------------------------------------------------------------
> reporte("Conveniencia", muestra_conv, parametro)
[Conveniencia] n=20  media=1348.50 USD  (parámetro 1330.72, diferencia +17.78)
   composición por ciudad:
Quito
   20
> reporte("Aleatorio simple", muestra_aas, parametro)
[Aleatorio simple] n=20  media=1318.00 USD  (parámetro 1330.72, diferencia -12.72)
   composición por ciudad:
  Cuenca Guayaquil     Quito
       5        7         8
> reporte("Estratificado", muestra_est, parametro)
[Estratificado] n=20  media=1226.00 USD  (parámetro 1330.72, diferencia -104.72)
   composición por ciudad:
  Cuenca Guayaquil     Quito
       4        6        10
>
> if (!is.null(muestra_conv) && !is.null(muestra_aas) && !is.null(muestra_est)) {
+     resumen <- data.frame(
+         diseno = c("Conveniencia (head)", "Aleatorio simple (sample)", "Estratificado (slice_sample)"),
+         n = c(nrow(muestra_conv), nrow(muestra_aas), nrow(muestra_est)),
+         media_estimada = round(c(mean(muestra_conv$ingresos_mensuales),
+                                  mean(muestra_aas$ingresos_mensuales),
+                                  mean(muestra_est$ingresos_mensuales)), 2)
+     )
+     cat(sprintf("Parámetro poblacional: %.2f USD\n", parametro))
+     print(resumen, row.names = FALSE)
+     cat("Con estas salidas responde en informe.md las preguntas P2 a P5:\n")
+     cat("  P2: ¿cuál diseño quedó más cerca del parámetro y por qué un solo\n")
+     cat("      número no basta para declarar un ganador?\n")
+     cat("  P3: ¿qué defecto tiene la muestra por conveniencia y cómo lo delata\n")
+     cat("      su composición por ciudad? ¿Se corrige con más filas?\n")
+     cat("  P4: ¿qué implica la variabilidad entre semillas para una decisión\n")
+     cat("      tomada con un solo estudio?\n")
+     cat("  P5: ¿cuándo usarías cada diseño? (piloto · subgrupos · conclusión)\n")
+ } else {
+     cat("Completa los TODO 1 a 4 para ver la tabla comparativa.\n")
+ }
Parámetro poblacional: 1330.72 USD
                   diseno  n media_estimada
       Conveniencia (head) 20        1348.5
   Aleatorio simple (sample) 20        1318.0
Estratificado (slice_sample) 20        1226.0
```

Figura 10. Posit Cloud: estratificado con slice_sample y comparación final en R.

Respuestas a las preguntas de la Parte B

P2. ¿Cuánto vale el parámetro y cuánto estimó cada diseño? ¿Cuál quedó más cerca y por qué eso no alcanza para declarar un ganador?

El parámetro es 1330.72 USD. La conveniencia estimó 1348.50 (+17.78), el aleatorio simple 1336.00 (+5.28) y el estratificado 1287.50 (−43.22). El más cercano fue el aleatorio simple, pero cada diseño se probó con un solo sorteo: con la semilla 3 el mismo diseño quedó a −95.72 y con la 4 a +160.78 USD, y en R el estratificado quedó a −104.72. La conveniencia quedó cerca solo por casualidad, ya que su muestra es 100 % de Quito y siempre sale igual. Un diseño se juzga por lo que garantiza y por cómo se comporta al repetirse, no por un único resultado.

P3. La muestra por conveniencia también tiene 20 filas. ¿Qué defecto tiene, cómo lo delata la composición por ciudad y se corrige con más filas?

Tiene sesgo de selección: el archivo está ordenado por ciudad y head(20) no sortea, solo toma el inicio. La composición lo delata: Quito 20 en Python y en R, cuando Quito es solo el 50 % de la población; Guayaquil (30 %) y Cuenca (20 %) quedaron fuera. No se corrige con más filas: con el archivo ordenado así, head(40) o head(100) seguirían trayendo solo Quito. Es un error sistemático que se repite igual cada vez; lo que lo corrige es cambiar el diseño.

P4. ¿Qué implica la variabilidad entre semillas para una empresa que decide con un solo estudio?

Las cinco medias en Python fueron 1336.00, 1405.00, 1235.00, 1491.50 y 1288.00 USD (rango 256.50; en R 152.50). Con un solo estudio la empresa recibe solo una de esas respuestas sin saber cuál, y podría concluir 1235 o 1491 USD por puro azar. A diferencia del sesgo, esta variabilidad viene del sorteo y se reduce con muestras más grandes. Por eso la estimación debe reportarse con su margen de incertidumbre y no conviene decidir con un solo número de una muestra de 20.

P5. ¿Cuándo usarías cada diseño en la práctica?

La conveniencia, solo para un piloto, como probar si una encuesta se entiende con los primeros clientes que llegan, sin generalizar. El aleatorio simple, para concluir sobre toda la población cuando no hay subgrupos obligatorios. El estratificado, cuando los subgrupos deben aparecer, como evaluar una campaña por ciudad: garantiza 10 de Quito, 6 de Guayaquil y 4 de Cuenca, algo que el aleatorio simple no asegura.

4. ¿Qué aprendí con esta práctica?

Ahora entiendo que el tipo que asigna el lenguaje no es la clasificación de la variable. Antes habría tratado satisfaccion como número porque llega como int64 y le habría sacado el promedio; ahora primero me pregunto qué representa el dato y luego lo declaro con CategoricalDtype o un factor ordenado. También aprendí que con un solo error de

escritura, Postgrado, perdí dos registros sin aviso, así que desde ahora cuento los faltantes después de cada conversión.

Lo más difícil fue separar el sesgo de la variabilidad. La conveniencia quedó a solo 17.78 USD del parámetro y parecía buena; entendí que era un mal diseño cuando vi que los 20 eran de Quito. La próxima vez repetiría cada diseño muchas más veces (por ejemplo, 1000 sorteos) para ver la distribución completa de las medias en lugar de comparar un solo resultado.

5. Conclusiones

1. El dtype no clasifica la variable; el criterio sí. satisfacción llegó como int64 siendo ordinal y edad como entero siendo continua. Solo con el orden declarado min() devolvió Primaria y 2, y un nivel mal escrito generó 2 faltantes sin aviso.
2. El diseño decide la representatividad, no el tamaño. Las tres muestras tenían 20 filas, pero la de conveniencia fue 100 % de Quito y dejó fuera a la mitad de la base. Ese error es sistemático y no se arregla con más filas del mismo archivo ordenado.
3. Una sola corrida no basta para juzgar un diseño. El aleatorio simple quedó a +5.28 USD con la semilla 1, pero sus cinco sorteos variaron entre 1235.00 y 1491.50 USD; el estratificado quedó más lejos en este sorteo, pero es el único que garantiza la proporción 10/6/4 por ciudad.

Respuesta a la gerencia. Una muestra de 20 clientes sirve para estimar el ingreso mensual promedio (valor real: 1330.72 USD) siempre que se elija por sorteo, de preferencia estratificado por ciudad, y que la cifra se reporte con su margen de error. Esta conclusión aplica a la base de 400 clientes de Quito, Guayaquil y Cuenca, no a clientes fuera de ella ni a otras ciudades.

6. Bibliografía

Bruce, P., Bruce, A. y Gedeck, P. (2020). *Practical Statistics for Data Scientists* (2.ª ed.). O'Reilly Media.

Lohr, S. L. (2021). *Sampling: Design and Analysis* (3.ª ed.). Chapman and Hall/CRC.

pandas development team. (2026). *pandas.CategoricalDtype*. https://pandas.pydata.org/docs/reference/api/pandas.CategoricalDtype.html

Wickham, H., François, R., Henry, L., Müller, K. y Vaughan, D. (2023). *dplyr: A Grammar of Data Manipulation*. https://dplyr.tidyverse.org/

Escuela Politécnica Nacional. (2026). *Laboratorio: clasificación de variables y población vs. muestra* [Repositorio y datos clientes.csv y clientes_400.csv]. IDSD422, 2026-B.

Declaración de uso de IA. Usé Claude (Anthropic) como apoyo para ordenar la redacción y darle el formato de la plantilla. El código, las ejecuciones en Colab y Posit Cloud y las capturas son de mi propia ejecución, y verifiqué que cada cifra coincida con las salidas.